



Lit - Cheat Sheet (Guía Profesional)

Stack: TypeScript, Lit 3.x, Vite, Web Components.

Índice

- [1. Inicialización Rápida](#)
 - [2. Decoradores Principales \(TypeScript\)](#)
 - [3. Template Syntax \(lit-html\)](#)
 - [4. Flujo de Control Estándar](#)
 - [5. Arquitectura: Flujo Unidireccional](#)
 - [6. Directivas de Estilos Dinámicos](#)
 - [7. Ciclo de Vida del Web Component](#)
 - [8. Rendimiento: Renderizado Indexado \(repeat\)](#)
 - [9. Flujo de Control Funcional Avanzado](#)
 - [10. Micro-optimizaciones del DOM](#)
 - [11. Context API \(@lit/context\)](#)
 - [12. Shadow DOM y Proyección \(Slots\)](#)
 - [13. Controladores Reactivos](#)
 - [14. Formularios y Extracción de Datos](#)
-

1. Inicialización Rápida

 [Leer en lit.dev](#)

Creación de un proyecto moderno usando el template oficial de TypeScript.

```
npm create vite@latest my-lit-app -- --template lit-ts  
cd my-lit-app  
npm install  
npm run dev
```

2. Decoradores Principales (TypeScript)

 Leer en lit.dev

Imports desde `lit/decorators.js`. Lit utiliza el estándar de decoradores de TypeScript para simplificar la creación de Web Components.

Decorador	Descripción
<code>@customElement('my-tag')</code>	Registra la clase como un Custom Element nativo en el DOM del navegador.
<code>@property({ type: Type })</code>	Define una propiedad pública reactiva (API del componente). Acepta String, Number, Boolean, Object, Array. Si su valor cambia, el componente se re-renderiza.
<code>@state()</code>	Define un estado interno, privado y reactivo de la clase. Altera la vista cuando cambia.
<code>@query('#my-id')</code>	Obtiene una referencia asíncrona, segura y tipada a un nodo interno del Shadow DOM.

 **Tip de Mutabilidad:** Lit observa los cambios por referencia. Si tu `@state()` o `@property()` es un **Array u Objeto**, mutarlo directamente (ej. `this.arr.push(1)`) **NO** repintará la pantalla. Debes asignar una nueva referencia (`this.arr = [...this.arr, 1]`) o llamar manualmente al método `this.requestUpdate()`.

3. Template Syntax (lit-html)

[Leer en lit.dev](#)

Enlaza los datos de la clase TypeScript al HTML de forma puramente declarativa dentro de la función `render()`.

- **Data Binding (Texto):** Interpolación directa de valores.

```
html`<p>Hola, ${this.nombre}</p>`
```

- **Property Binding (.):** Para pasar estructuras de datos complejas (objetos, arrays) hacia otros Web Components anidados.

```
html`<mi-componente .usuario=${this.userObj}></mi-componente>`
```

- **Attribute Binding (sin prefijo):** Para inyectar valores en atributos HTML nativos de tipo string o numérico.

```
html`<div id=${this.dynamicId}></div>`
```

- **Boolean Attribute (?):** Inserta o elimina un atributo en el DOM dinámicamente basado en una evaluación booleana (ideal para `disabled`, `readonly`, `hidden`).

```
html`<button ?disabled=${this.isLoading}>Enviar</button>`
```

- **Event Binding (@):** Adjunta un *Event Listener* estándar (nativos o CustomEvents) al elemento.

```
html`<button @click=${this.handleClick}>Click</button>`
```

4. Flujo de Control Estándar

 Leer en lit.dev

Lit no usa motores de plantillas pesados; delega toda la lógica de control a la sintaxis nativa de JavaScript/TypeScript.

Condicionales (If / Else)

Tienes dos formas principales según la complejidad:

- **En línea (Ternario):** Ideal para alternar bloques simples.

```
html`${this.hasAccess ? html`<button>Entrar</button>` :  
html`<p>Denegado</p>`}`
```

- **Bloque clásico (Fuera del HTML):** Si la condición es muy compleja, usa un `if` nativo antes de retornar.

```
render() {  
  if (this.isLoading) return html`<loading-spinner></loading-spinner>`;  
  return html`<main-content></main-content>`;  
}
```

Bucles y Listas (For)

Tampoco hay directivas especiales, se manipulan arrays de forma nativa:

- **Mapecto en línea (`.map`):** La forma estándar.

```
html`  
  <ul>  
    ${this.items.map((item: Item) => html`<li>${item.name}</li>`)}  
  </ul>  
`
```

- **Bucle clásico (`for...of`):** Útil si necesitas filtrar u operar con lógica antes de pintar.

```
render() {  
  const listHtml = [];  
  for (const item of this.items) {  
    if (item.isActive) listHtml.push(html`<li>${item.name}</li>`);  
  }  
  return html`<ul>${listHtml}</ul>`;  
}
```

↕ 5. Arquitectura: Flujo Unidireccional

🔗 Leer en lit.dev

La regla de oro para la comunicación entre componentes Web es: **Propiedades hacia abajo, Eventos hacia arriba.**

↓ Hacia los Hijos (Inyección de Propiedades)

El componente superior expone y transmite los datos usando la sintaxis de punto (`.`).

```
// En el componente Superior (Padre):
html`<item-card .item=${this.miDato}></item-card>`

// En el componente Inferior (item-card.ts):
@property({ type: Object }) item?: Item;
```

↑ Hacia los Padres (Despacho de Eventos)

El componente inferior emite eventos personalizados usando la API nativa `CustomEvent`, configurada para traspasar el límite del Shadow DOM.

```
// En el componente Inferior (Despacha):
private seleccionar(): void {
  this.dispatchEvent(new CustomEvent('item-seleccionado', {
    detail: { id: this.item?.id },
    bubbles: true,
    composed: true // Atraviesa el Shadow DOM
  }));
}

// En el componente Superior (Escucha):
html`<item-card @item-seleccionado=${this.manejarSeleccion}></item-card>`

private manejarSeleccion(event: CustomEvent<{id: string}>): void {
  console.log("Elemento seleccionado ID:", event.detail.id);
}
```

6. Directivas de Estilos Dinámicos

 Leer en lit.dev

Imports desde `lit/directives/...`

- **classMap**: Evalúa y aplica clases CSS al elemento de manera reactiva y limpia.

```
import { classMap } from 'lit/directives/class-map.js';

const classes = { active: this.isActive, error: this.hasError };
html`<div class=${classMap(classes)}></div>`
```

- **styleMap**: Genera estilos en línea (inline-styles) reaccionando al estado local.

```
import { styleMap } from 'lit/directives/style-map.js';

const styles = { color: 'red', marginTop: '10px' };
html`<div style=${styleMap(styles)}></div>`
```

7. Ciclo de Vida del Web Component

 Leer en lit.dev

Métodos base heredados de `LitElement` que facilitan enganchar lógica en puntos deterministas del renderizado.

Método	Descripción y Casos de Uso
<code>connectedCallback()</code>	El elemento se ha añadido físicamente al DOM. Instante recomendado para arrancar <code>setInterval</code> , adjuntar <code>addEventListener</code> a <code>window</code> o iniciar peticiones asíncronas. Es imperativo llamar a <code>super.connectedCallback()</code>.
<code>firstUpdated()</code>	Lit ha completado el primer renderizado. El árbol HTML local está disponible; seguro para leer medidas físicas, canvas o instanciar librerías de terceros (ej. Chart.js) seleccionando nodos con <code>@query</code> .
<code>updated(changedProps)</code>	Se invoca tras cada ciclo de repintado exitoso. Contiene un <code>Map</code> para revisar si mutó una propiedad en concreto frente a su valor anterior.
<code>disconnectedCallback()</code>	El elemento se ha desmontado del DOM. Obligatorio para recolección de basura (limpiar event listeners remotos, limpiar timers) y evitar <i>memory leaks</i> .

8. Rendimiento: Renderizado Indexado (repeat)

 Leer en lit.dev

Para evitar los cuellos de botella del `.map()` estándar (que destruye y reconstruye todos los nodos DOM si cambia el índice o el orden), la directiva `repeat()` asocia nodos a una llave única para moverlos en lugar de borrarlos.

```
import { repeat } from 'lit/directives/repeat.js';

html`
  <ul>
    ${repeat(
      this.items,
      (item: Item) => item.id, // Identity Key (immutable)
      (item: Item, index: number) => html`<li>${index}: ${item.name}</li>`
    )}
  </ul>
`
```

9. Flujo de Control Funcional Avanzado

[Leer en lit.dev](#)

Lit provee directivas declarativas para simplificar ternarios anidados y agilizar el motor del DOM virtual.

- **Cache:** Extrae el fragmento condicional inactivo y lo pausa en memoria en lugar de destruirlo físicamente. Óptimo al alternar componentes complejos como vistas o pestañas para garantizar recargas instantáneas.

```
import { cache } from 'lit/directives/cache.js';

html`${cache(this.vista === 'lista' ? html`<vista-lista></vista-lista>` :
html`<vista-grid></vista-grid>`)}`
```

- **When:** Directiva sintáctica que reemplaza las expresiones ternarias aportando legibilidad al *early return*.

```
import { when } from 'lit/directives/when.js';

html`${when(
  this.user,
  () => html`<p>Bienvenido ${this.user!.name}</p>`,
  () => html`<p>Sesión expirada</p>`
)}`
```

10. Micro-optimizaciones del DOM

 Leer en lit.dev

- **willUpdate(changedProperties)** : Se acciona *antes* de la fase síncrona de `render()` . Es el escenario adecuado para la computación de estados derivados o formatters pesados (ej. sumas totales o transformación de strings). Modificar estados aquí **no disparará un nuevo ciclo de renderizado**.

```
protected willUpdate(changedProperties: PropertyValues<this>): void {  
  if (changedProperties.has('precioBase') ||  
      changedProperties.has('impuestos')) {  
    this.precioTotal = this.precioBase + this.impuestos; // 1 solo repaint  
  }  
}
```

- **await this.updateComplete** : Promesa arquitectónica del motor Lit. Permite pausar la ejecución de una función asíncrona hasta asegurar que las actualizaciones de estado recientes han terminado de pintarse físicamente en la pantalla.

```
async abrirDialogo(): Promise<void> {  
  this.dialogoVisible = true;  
  await this.updateComplete; // Garantiza que el nodo HTML existe  
  this.dialogElement?.showModal();  
}
```



11. Context API (@lit/context)

[Leer en lit.dev](#)

Herramienta de Inyección de Dependencias. Soluciona el problema de paso en cascada (*Prop Drilling*), distribuyendo contextos, configuraciones o variables globales sin conectarlos manualmente jerarquía por jerarquía.

```
import { createContext, provide, consume } from '@lit/context';
import type { UserContext } from './types';

// 1. Declarar Token Simbólico
export const userContext = createContext<UserContext>('user-context');

// 2. Elemento Proveedor (Módulo superior/Padre)
@customElement('app-root')
export class AppRoot extends LitElement {
  @provide({ context: userContext })
  @state() user: UserContext = { name: 'Admin', role: 'admin' };
}

// 3. Elemento Consumidor (Descendientes en cualquier capa)
@customElement('user-profile')
export class UserProfile extends LitElement {
  @consume({ context: userContext, subscribe: true })
  @state() user!: UserContext;
}
```

12. Shadow DOM y Proyección (Slots)

[Leer en lit.dev](#)

Patrón para crear componentes contenedores reutilizables inyectando HTML desde el componente padre.

```
<!-- Instanciación e Inyección externa -->
<tarjeta-informativa>
  <h2 slot="titulo">Alerta Global</h2>
  <p>Texto distribuido hacia el default slot de la tarjeta.</p>
</tarjeta-informativa>
```

```
// Estructura interna de tarjeta-informativa.ts
render() {
  return html`
    <header>
      <slot name="titulo">Título de respaldo</slot>
    </header>
    <main>
      <slot></slot> <!-- Receptor HTML genérico -->
    </main>
  `;
}
```

- **Estilizado cruzado de los Slots:** Como el Shadow DOM garantiza total encapsulación, el único puente de comunicación CSS permitido hacia los elementos proyectados es el pseudo-elemento especial

`::slotted()` .

```
::slotted(h2) {
  color: var(--primary-color);
  font-weight: bold;
  margin: 0;
}
```



13. Controladores Reactivos

[Leer en lit.dev](#)

Patrón arquitectónico de Lit para extraer, encapsular y reutilizar la lógica de estado o ciclo de vida (ej. Fetching de APIs, subscripción a Websockets o Timers recurrentes) sin ensuciar la clase visual del componente.

```
import { ReactiveController, ReactiveControllerHost } from 'lit';

// 1. Entidad Controladora Aislada
export class RelojLogicoController implements ReactiveController {
  private timer?: number;
  public valor: Date = new Date();

  constructor(private host: ReactiveControllerHost) {
    this.host.addController(this); // Registrar la vinculación del ciclo
  }

  hostConnected(): void {
    this.timer = window.setInterval(() => {
      this.valor = new Date();
      this.host.requestUpdate(); // Emite señal síncrona de renderizado al Host
    }, 1000);
  }

  hostDisconnected(): void {
    clearInterval(this.timer); // Prevención de Fugas de memoria
  }
}

// 2. Composición de UI (Múltiples componentes pueden rehusarlo)
@customElement('display-reloj')
export class DisplayReloj extends LitElement {
  private clock = new RelojLogicoController(this); // Instancia pura

  render() {
    return html`
      <div class="clock-ui">
        Tiempo Local: <strong>${this.clock.valor.toLocaleTimeString()}</strong>
      </div>
    `;
  }
}
```

14. Formularios y Extracción de Datos

[Leer sobre eventos en lit.dev](#)

Lit **no** tiene un sistema propio de formularios complejos. Abraza la **plataforma Web**, utilizando elementos `<form>` nativos y la API estándar `FormData`. Todo es asombrosamente sencillo si conoces JavaScript moderno.

Recolección Nativa con FormData (Recomendado)

Para recolectar la información al enviar, atrapa el evento `@submit`, cancela la recarga y usa `FormData`.

```
@customElement('form-registro')
export class FormRegistro extends LitElement {
  private _handleSubmit(e: Event) {
    e.preventDefault(); // Impide recargar
    const dataObj = Object.fromEntries(new FormData(e.target as
    HTMLFormElement).entries());
    console.log("Datos listos:", dataObj);
  }
  render() {
    return html`
      <form @submit=${this._handleSubmit}>
        <!-- IMPORTANTÍSIMO: usar atributo "name" -->
        <input type="text" name="usr" required />
        <button type="submit">Enviar</button>
      </form>`;
  }
}
```

Two-Way Data Binding Manual

Lit implementa flujo *unidireccional*. Para actualizar variables en tiempo real a cada pulsación, escucha `@input`.

```
@customElement('buscador-en-vivo')
export class BuscadorEnVivo extends LitElement {
  @state() private t: string = "";
  private _onInput(e: Event) {
    this.t = (e.target as HTMLInputElement).value; // Dispara re-render
  }
  render() {
    return html`
      <input type="text" .value=${this.t} @input=${this._onInput} />
      <p>Buscando: <strong>${this.t}</strong></p>`;
  }
}
```